

Spring Boot RESTful API - Practical Questions

Module 1-3: Introduction to Spring Boot & Building REST Controllers

Instructions

- Create a new Spring Boot project using Spring Initializr (start.spring.io)
 - Add only the **Spring Web** dependency
 - Test your APIs using Postman or browser
 - Focus on creating REST Controllers only (no service or repository layers needed)
 - Use proper HTTP methods and status codes
-

Question 1: Library Book Management API (20 Points)

Create a RESTful API for a simple library book management system.

Requirements:

1. Create a **Book** class with the following attributes:
 - o id (Long)
 - o title (String)
 - o author (String)
 - o isbn (String)
 - o publicationYear (int)
2. Create a **BookController** with the following endpoints:
 - o **GET** /api/books - Return a list of all books (create 3 sample books in a List)
 - o **GET** /api/books/{id} - Return a specific book by ID
 - o **GET** /api/books/search?title={title} - Search books by title
 - o **POST** /api/books - Add a new book (accept Book object in request body)
 - o **DELETE** /api/books/{id} - Delete a book by ID

Expected Output Examples:

GET /api/books

```
[
  {
    "id": 1,
    "title": "Clean Code",
    "author": "Robert Martin",
```

```
"isbn": "978-0132350884",  
"publicationYear": 2008  
}  
]
```

Bonus: Add appropriate HTTP status codes (200, 201, 204, 404)

Question 2: Student Registration API (20 Points)

Build a REST API for student registration and information management.

Requirements:

3. Create a **Student** class with:
 - studentId (Long)
 - firstName (String)
 - lastName (String)
 - email (String)
 - major (String)
 - gpa (Double)
4. Create a **StudentController** with these endpoints:
 - **GET** /api/students - Get all students
 - **GET** /api/students/{studentId} - Get student by ID
 - **GET** /api/students/major/{major} - Get all students by major (use path variable)
 - **GET** /api/students/filter?gpa={minGpa} - Filter students with GPA greater than or equal to minimum
 - **POST** /api/students - Register a new student
 - **PUT** /api/students/{studentId} - Update student information

Testing Scenarios:

- Create at least 5 sample students with different majors and GPAs
 - Test filtering students by Computer Science major
 - Test filtering students with GPA ≥ 3.5
-

Question 3: Restaurant Menu API (20 Points)

Create an API for a restaurant menu management system.

Requirements:

1. Create a **MenuItem** class with:
 - id (Long)
 - name (String)
 - description (String)
 - price (Double)
 - category (String) - e.g., "Appetizer", "Main Course", "Dessert", "Beverage"
 - available (boolean)
2. Create a **MenuController** with endpoints:
 - **GET** /api/menu - Get all menu items
 - **GET** /api/menu/{id} - Get specific menu item
 - **GET** /api/menu/category/{category} - Get items by category
 - **GET** /api/menu/available - Get only available items (use query parameter: ?available=true)
 - **GET** /api/menu/search?name={name} - Search menu items by name
 - **POST** /api/menu - Add new menu item
 - **PUT** /api/menu/{id}/availability - Toggle item availability
 - **DELETE** /api/menu/{id} - Remove menu item

Challenge: Create at least 8 different menu items across all categories

Question 4: E-Commerce Product API (25 Points)

Develop a REST API for an e-commerce product catalog.

Requirements:

3. Create a **Product** class with:
 - productId (Long)
 - name (String)
 - description (String)
 - price (Double)
 - category (String)
 - stockQuantity (int)

- brand (String)
- 4. Create a **ProductController** with these endpoints:
 - **GET** /api/products - Get all products (with optional pagination params: ?page={page}&limit={limit})
 - **GET** /api/products/{productId} - Get product details
 - **GET** /api/products/category/{category} - Get products by category
 - **GET** /api/products/brand/{brand} - Get products by brand
 - **GET** /api/products/search?keyword={keyword} - Search products by keyword in name or description
 - **GET** /api/products/price-range?min={min}&max={max} - Get products within price range
 - **GET** /api/products/in-stock - Get products with stockQuantity > 0
 - **POST** /api/products - Add new product
 - **PUT** /api/products/{productId} - Update product details
 - **PATCH** /api/products/{productId}/stock?quantity={quantity} - Update stock quantity
 - **DELETE** /api/products/{productId} - Delete product

Testing Requirements:

- Create at least 10 products with different categories, brands, and prices
- Test all search and filter functionalities
- Use appropriate HTTP status codes

Question 5: Task Management API (15 Points)

Build a simple task/to-do list API.

Requirements:

1. Create a **Task** class with:
 - taskId (Long)
 - title (String)
 - description (String)
 - completed (boolean)
 - priority (String) - "LOW", "MEDIUM", "HIGH"
 - dueDate (String) - format: "YYYY-MM-DD"

2. Create a **TaskController** with:

- **GET** `/api/tasks` - Get all tasks
- **GET** `/api/tasks/{taskId}` - Get task by ID
- **GET** `/api/tasks/status?completed={true/false}` - Get tasks by completion status
- **GET** `/api/tasks/priority/{priority}` - Get tasks by priority
- **POST** `/api/tasks` - Create new task
- **PUT** `/api/tasks/{taskId}` - Update task
- **PATCH** `/api/tasks/{taskId}/complete` - Mark task as completed
- **DELETE** `/api/tasks/{taskId}` - Delete task

Bonus Question: User Profile API (Extra 20 Points)

Create a comprehensive user profile management API.

Requirements:

3. Create a **UserProfile** class with:

- `userId` (Long)
- `username` (String)
- `email` (String)
- `fullName` (String)
- `age` (int)
- `country` (String)
- `bio` (String)
- `active` (boolean)

4. Create a **UserProfileController** with multiple endpoints including:

- Basic CRUD operations
- Search by username, country, or age range
- Activate/deactivate user profiles
- Return custom response messages wrapped in a response object

Response Wrapper Example:

```
public class ApiResponse<T> {  
    private boolean success;  
    private String message;  
    private T data;  
}
```

```
// constructors, getters, setters  
}
```

Return responses like:

```
{  
  "success": true,  
  "message": "User profile created successfully",  
  "data": {  
    "userId": 1,  
    "username": "john_doe",  
    "email": "john@example.com"  
  }  
}
```

Submission Guidelines

5. **Project Structure:** Each question should be a separate Spring Boot project
6. **Naming Convention:** Name your projects as `question1-library-api`, `question2-student-api`, etc.
7. **Documentation:** Include a README.md file with:
 - List of all endpoints
 - Sample request/response for each endpoint
 - How to run the application
8. **Testing:** Include screenshots or Postman collection showing successful API calls
9. **Code Quality:**
 - Use meaningful variable names
 - Add comments where necessary
 - Follow Java naming conventions
 - Proper indentation

Grading Criteria

- **Correct Implementation** (60%): All endpoints work as specified
 - **Code Quality** (20%): Clean, readable, well-organized code
 - **HTTP Methods & Status Codes** (10%): Proper use of GET, POST, PUT, DELETE, PATCH
 - **Testing** (10%): Evidence of testing all endpoints
-

Tips for Success

1. Start with Question 1 and progress sequentially
2. Test each endpoint immediately after creating it
3. Use `@PathVariable` for URL parameters like `/api/books/{id}`
4. Use `@RequestParam` for query parameters like `/api/books?title=clean`
5. Use `@RequestBody` for POST/PUT requests
6. Remember to use appropriate annotations:
 - `@RestController`
 - `@RequestMapping`
 - `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@PatchMapping`
 - `@PathVariable`
 - `@RequestParam`
7. Initialize sample data using a List or ArrayList in your controller
8. Test your APIs using Postman before submission
9. Refer to the shared project so that you can use it. You need to have two main packages (controller and model) and each question has its inner package. ex: the controller for ecommerce should be in package ecommerce, then the class Product should be in modal under ecommerce. NB: if you want to use the shared project, you will need to change to the jdk you have in your PC.

```
29  <properties>
30    <java.version>17</java.version>
31  </properties>
```

Submission of the assignment:

Create a branch with name: restFull_api_StudentId
Push the codes on that branch.

Deadline is next class at 17:59 before the class.

Good Luck!